

CIRCLE CI



PRODUCTION

HANDBOOK



FROM CI/CD FUNDAMENTALS
TO PRODUCTION DELIVERY



CODE
COMMIT



CIRCLECI
PIPELINE



BUILD



TEST



SECURITY
SCAN



PACKAGE &
PUBLISH



DEPLOY



VERIFY

ENGINEERED FOR PRODUCTION

- ✓ Reliable CI/CD Pipelines
- ✓ Secure by Design
- ✓ Fast, Scalable & Efficient
- ✓ Built for Real-World Delivery
- ✓ From Code to Production

WHAT YOU WILL MASTER

- CircleCI Core Concepts
- Pipeline Design & Orchestration
- Orbs, Caching & Workspaces
- Testing, Quality & Security
- Docker, Kubernetes & Cloud Deployments
- Performance & Cost Optimization
- Troubleshooting & Best Practices



PRODUCTION READY SKILLS

- ✓ Secure Pipelines
Secrets, Contexts, Least Privilege
- ✓ High Performance
Parallelism, Caching, Optimization
- ✓ Cloud Native Deployments
AWS, Kubernetes, Docker
- ✓ Observability & Reliability
Logs, Metrics, Rollbacks



PRODUCTION FOCUS

- ✓ Real-World Scenarios
- ✓ Best Practices
- ✓ Common Failures & Fixes
- ✓ Security Hardening
- ✓ Scalability & Reliability
- ✓ Enterprise-Grade Delivery



DESIGNED BY
ENGINEERS
For Engineers



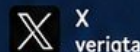
PRACTICAL
& ACTIONABLE
No Fluff



EXAMPLES
YOU CAN USE
In Real Projects



LEARN TODAY
DELIVER
Tomorrow



1. CIRCLECI ARCHITECTURE AND CORE CONCEPTS

✓ WHAT CIRCLECI SOLVES

- Automates the entire software delivery process.
- Eliminates manual builds, tests and deployments.
- Provides fast, consistent and repeatable CI/CD pipelines.
- Supports modern DevOps with scalability and reliability.
- Integrates deeply with GitHub, GitLab, Bitbucket and more.
- Helps teams ship code faster and safer to production.

🔄 PIPELINES, WORKFLOWS, JOBS, AND STEPS



PIPELINE: The full CI/CD process triggered by an event (push, PR, tag, schedule, etc.).



WORKFLOW: A collection of one or more jobs that define the structure and order of execution.



JOB: A set of steps that run in the same environment (container or VM).



STEP: An individual action within a job. (e.g., run command, checkout code, install deps).

✓ CLOUD VERSUS SELF-HOSTED EXECUTION



CLOUD EXECUTION (CIRCLECI CLOUD)

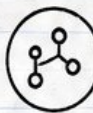
- No infrastructure to manage.
- Scales automatically.
- Pronaged by CircleCI.
- Pay for usage (minutes).
- Best for most teams.



SELF-HOSTED EXECUTION (CIRCLECI SERVER)

- Runs on your own infrastructure.
- Full control and data residency.
- No usage-based pricing.
- You manage scaling and upgrades.
- Best for regulated environments.

🔄 PIPELINE LIFECYCLE



1. TRIGGER

Event occurs (push, PR, schedule, etc.)



2. QUEUED

Pipeline is queued and resources are allocated.



3. RUNNING

Jobs and steps execute in defined order/parallel.



4. COMPLETED

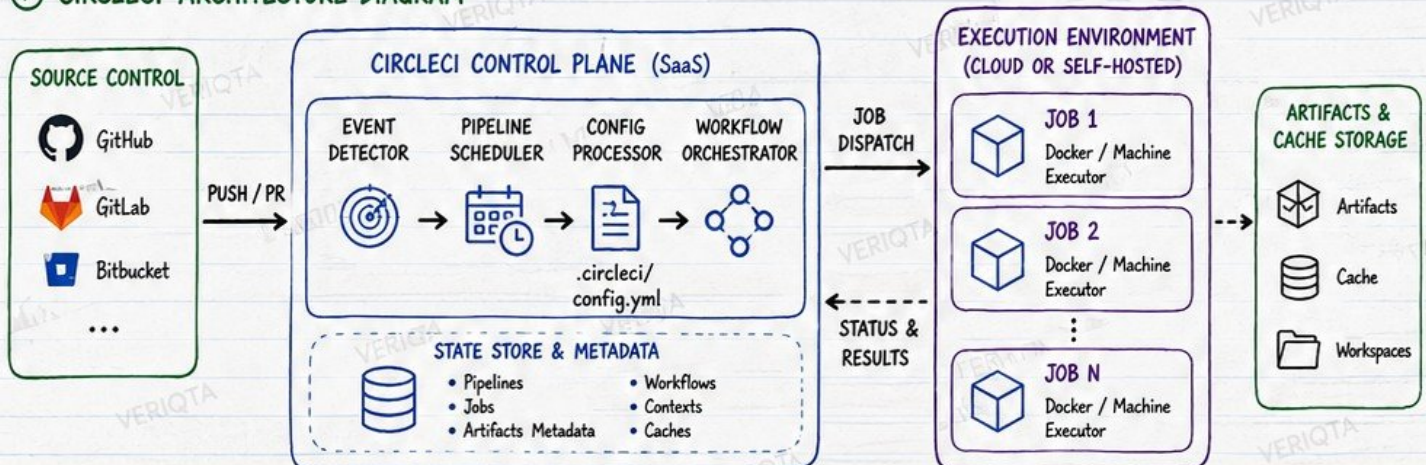
Pipeline succeeds or fails.



5. NOTIFICATION

Results are sent to team (email, Slack, Webhooks).

✓ CIRCLECI ARCHITECTURE DIAGRAM



KEY POINTS

- ✓ CircleCI is event-driven and Git-centric.
- ✓ Workflows define the structure; jobs do the work.
- ✓ Execution happens in isolated, reproducible environments.
- ✓ Artifacts, caches and workspaces speed up delivery.
- ✓ Everything is defined as code in `.circleci/config.yml`



PRODUCTION NOTE

A well-designed pipeline is fast, predictable and observable. Optimize for feedback speed, reliability and security.



SENIOR ENGINEER TIP

Keep jobs small and focused. Run independent jobs in parallel. Use caching and workspaces wisely. Fail fast, notify quickly, and automate recovery.



YOUTUBE
veriqta



GITHUB
veriqta



LINKEDIN
veriqta



X
veriqta



INSTAGRAM
veriqta



TELEGRAM
veriqta

2. UNDERSTANDING .CIRCLECI/CONFIG.YML

The entire pipeline is defined as code in one YAML file located in the .circleci/config.yml

A. CONFIGURATION STRUCTURE

A CircleCI config file is written in YAML.

It describes **WHAT** to run, **HOW** to run it, and **WHEN** to run it.

-  **version**
Specifies the CircleCI configuration version.
-  **executors**
Defines the environments where jobs run.
-  **jobs**
A collection of steps executed in the same environment.
-  **workflows**
Orchestrates jobs and defines their order or parallelism.

B. VERSION DECLARATION

The version key is mandatory.

It tells CircleCI which configuration syntax to use.

```
version: 2.1
```

Currently the stable and recommended version.

C. EXECUTORS

Executors define the environment for jobs.

You can use built-in or custom executors.

```
executors:
  docker-executor:
    docker:
      - image: cimg/node:20.11

  machine-executor:
    machine:
      image: ubuntu-2204:current
      resource_class: medium
```

Define once, reuse in multiple jobs.

E. WORKFLOWS

Workflows define which jobs run and in what order.

```
workflows:
  version: 2
  build_and_test:
    jobs:
      - build
      - deploy:
          requires:
            - build
```

The deploy job runs only after the build job succeeds.

D. JOBS AND STEPS

Jobs are made of ordered steps. Each step is a single action.

```
1 version: 2.1
2
3 executors:
4   node-executor:
5     docker:
6       - image: cimg/node:20.11
7
8 jobs:
9   build:
10    executor: node-executor
11    steps:
12      - checkout
13      - run:
14        name: Install Dependencies
15        command: npm ci
16      - run:
17        name: Run Tests
18        command: npm test
19      - persist_to_workspace:
20        root: .
21        paths:
22          - node_modules
23
24 workflows:
25   build_and_test:
26     jobs:
27       - build
```

Version of the config

Executors (environments)

Jobs (units of work)

Steps (actions inside a job)

Workflows (orchestration of jobs)

F. CONFIGURATION VALIDATION

Always validate before pushing changes.

1. Using CircleCI CLI

```
$ circleci config validate
```

✓ Config File is Valid!

2. Using Local Lint (no CLI)

```
$ circleci config process
```

Outputs the fully expanded config.

3. Common Validation Errors

- Indentation mistakes
- Wrong key names
- Invalid executor names
- Missing steps
- YAML syntax errors



SENIOR ENGINEER TIPS

- ✓ Keep configs DRY. Reuse executors and commands.
- ✓ Use meaningful job names.
- ✓ Keep steps small and focused.
- ✓ Fail fast to save CI minutes.
- ✓ Use workspaces for sharing build outputs.
- ✓ Validate locally before committing.
- ✓ Document complex logic in comments.



3. EXECUTORS AND EXECUTION ENVIRONMENTS

Executors define the environment where your jobs run.
CircleCI provides 3 built-in executors so you can choose the right environment for the right task.

A. BUILT-IN EXECUTORS



1. DOCKER EXECUTOR

- Runs your job in a Docker container.
- Fast startup (seconds).
- Ephemeral – clean environment for every job.
- Ideal for most CI/CD tasks: build, test, lint, package.

BEST FOR

Most projects, small to medium workloads.

example:

executors:

docker-executor:

docker:

- image: cimg/node:20.11



2. MACHINE EXECUTOR

- Runs your job on a real VM.
- Full VM access (root).
- Persistent for the duration of the job.
- Supports Docker-in-Docker, systemd, large builds, custom packages.

BEST FOR

Docker-in-Docker, large builds, system-level operations.

example:

executors:

machine-executor:

machine:

image: ubuntu-2204:current



3. MACOS EXECUTOR

- Runs your job on macOS machines hosted by CircleCI.
- Required for iOS/macOS builds, Xcode, signing, notarization.
- Limited availability and higher cost compared to Docker.

BEST FOR

iOS/macOS builds, Xcode, signing, UI tests.

example:

executors:

macos-executor:

macos:

xcode: "15.2.0"

B. EXECUTOR COMPARISON

FEATURE	DOCKER	MACHINE	MACOS
Isolation	High	Medium	High
Startup Time	Fast (secs)	Slow (mins)	Slow (mins)
Performance	Good	High	High
Root Access	No	Yes	Yes
Docker-in-Docker	Limited	Yes	N/A
Best For	Most CI tasks	Complex builds & system tasks	macOS/iOS builds
Cost	Low	Medium	High
Ephemeral	Yes	Yes	Yes



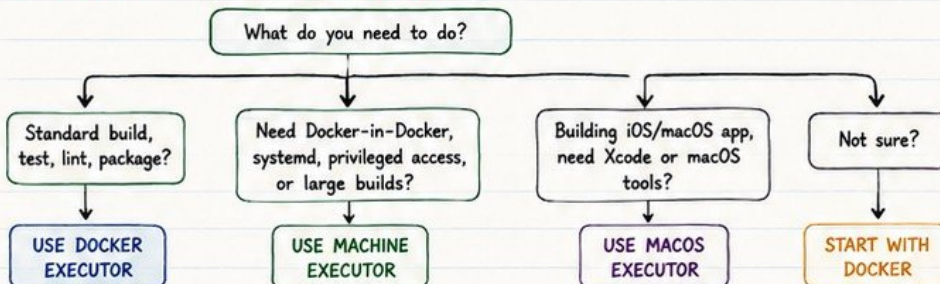
SENIOR ENGINEER TIP

Default to Docker executor.

Use Machine executor only when you absolutely need system-level control or Docker-in-Docker.

Use macOS executor only for Apple ecosystem builds.

C. CHOOSING THE CORRECT EXECUTOR



D. RESOURCE CLASSES (CPU & MEMORY)

Define how much CPU and RAM your job gets.

EXECUTOR	RESOURCE CLASS	vCPU	RAM	BEST USE CASE
Docker	small	1	2 GB	Light builds, tests
	medium	2	4 GB	Most projects (default)
	medium+	2	6 GB	Heavier builds, more memory
	large	4	8 GB	Parallel tests, large builds
	xlarge	8	16 GB	Very large builds, monorepos
Machine	medium	2	4 GB	General purpose
	large	4	8 GB	Larger builds, DB tests
	xlarge	8	16 GB	Big builds, integration suites
	2xlarge	16	32 GB	Very heavy workloads
MacOS	macos.m1.medium.gen1	2	4 GB	General macOS tasks
	macos.m1.large.gen1	4	8 GB	Xcode builds, UI tests

E. PRODUCTION TRADEOFFS



SPEED: Docker is fastest to spin up. Machine/macOS take longer.



COST: macOS is most expensive. Machine costs more than Docker.



ISOLATION: All executors are isolated. Avoid relying on state between jobs.



FLEXIBILITY: Machine gives full control. Use only when needed.



SCALABILITY: Docker scales best. Use smaller resource classes to control costs.



PRODUCTION NOTE

Over-provisioning wastes money.
Under-provisioning slows down pipelines.
Right-size your resource class based on real metrics.



YOUTUBE
veriqta



GITHUB
veriqta



LINKEDIN
veriqta



X
veriqta



INSTAGRAM
veriqta



TELEGRAM
veriqta

4. JOBS, STEPS, AND COMMANDS

The building blocks of every CircleCI pipeline.

4.1 JOB LIFECYCLE



PRODUCTION NOTE

- Every job runs in a clean, isolated environment.
- If any step fails (non-zero exit code), the job stops immediately.
- Design small, focused jobs for faster feedback and easier debugging.

4.3 JOB EXAMPLE

```
jobs:
  build-and-test:
    docker:
      - image: cimg/node:20.11
    working_directory: ~/project
    steps:
      - checkout
      - run:
          name: Install dependencies
          command: npm ci
      - run:
          name: Run tests
          command: npm test
      - run:
          name: Build
          command: npm run build
```

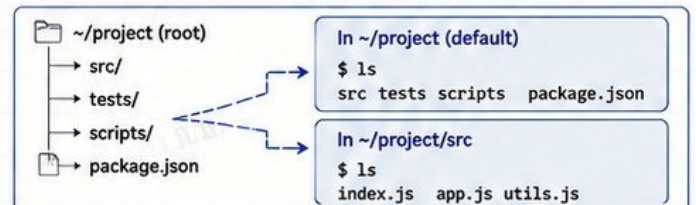
Steps run
in order
inside the
same job
container.

4.2 CORE STEPS EXPLAINED

	run Executes a shell command. If the command exits with non-zero code, the step (and job) fails. Example: <pre>- run: name: Install dependencies command: npm ci</pre>
	checkout Clones the repository into the workspace. Usually the first step in every job. Example: <pre>- checkout</pre>
	working_directory Sets the current directory for all following steps. Example: <pre>- working_directory: ~/project</pre>

4.4 WORKING DIRECTORIES

- Default directory after checkout: `~/project`
- Use `working_directory` to change it.
- Paths are relative to the working directory.



4.5 ENVIRONMENT VARIABLES

- Provided by CircleCI or defined in project settings.
- Access using `$VAR` or `${VAR}`.
- Useful for secrets, endpoints, feature flags, etc.
- Never hard-code secrets in config.yml.

SOURCE	EXAMPLE
Project / Context	<code>\$ DB_PASSWORD</code>
Built-in	<code>\$ CIRCLE_SHA1</code> , <code>\$ CIRCLE_BRANCH</code>
Set in job	<pre>- run: name: Set variable command: echo "export API_URL=https://a- api.example.com" >> \$BASH_ENV</pre>



SECURITY REMINDER

Use Contexts for secrets and limit access to trusted projects and branches.

4.6 EXIT CODES AND JOB FAILURES

- In Linux, every command returns an exit code.
- `0` = Success
- Non-zero = Failure
- CircleCI fails the step and job on any non-zero exit code.

EXIT CODE	MEANING	EXAMPLE
0	Success	Command completed OK
1	General error	Test failures, exceptions
2	Misuse of shell builtins	Incorrect command usage
126	Permission denied	Script not executable
127	Command not found	Typo or missing binary
130	Terminated (Ctrl+C)	Job cancelled manually
137	Killed (Out of Memory)	Container OOM killed



DEBUGGING INSIGHT

Add `set -euo pipefail` at the top of long scripts to fail fast on errors, undefined variables, and pipeline failures.



5. WORKFLOWS AND PIPELINE ORCHESTRATION

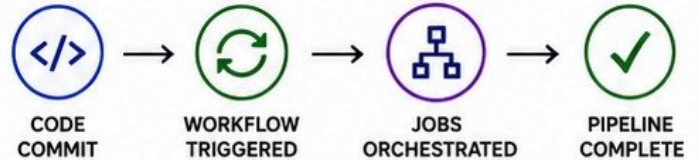
DESIGNING RELIABLE, FAST AND SCALABLE WORKFLOWS IN CIRCLECI

5.1 WHAT IS A WORKFLOW?

A workflow defines how jobs are orchestrated and in what order (or in parallel) they run. It controls the entire pipeline.

```
workflows:
  version: 2.1
  build_and_deploy:      # Workflow name
    jobs:
      - build
      - test:
          requires: [build]  # "test" runs after "build"
      - deploy:
          requires: [test]   # "deploy" runs after "test"
```

5.2 WORKFLOW ARCHITECTURE

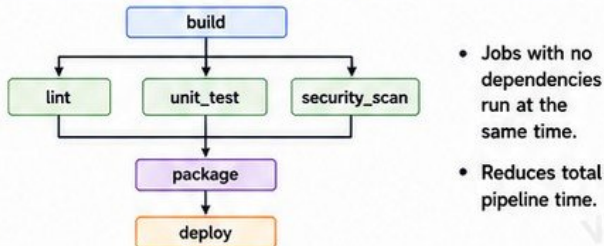


★ HOW IT WORKS

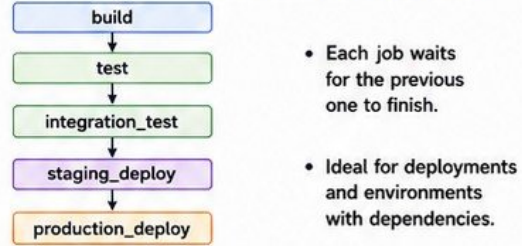
1. Code is pushed or a PR is opened.
2. Workflow is triggered.
3. Jobs are scheduled based on dependencies.
4. Jobs run in parallel or sequence.
5. When all required jobs succeed, workflow completes.

5.3 EXECUTION MODELS

PARALLEL EXECUTION (FASTER)



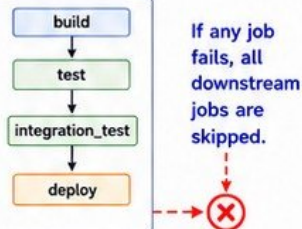
SEQUENTIAL EXECUTION (SAFE & ORDERED)



5.4 REQUIRES DEPENDENCIES

The `requires` keyword creates dependencies between jobs. A job will not start until all its required jobs succeed.

```
workflows:
  build_test_deploy:
    jobs:
      - build
      - test:
          requires: [build]
      - integration_test:
          requires: [test]
      - deploy:
          requires: [integration_test]
```



5.5 DESIGNING EFFICIENT PIPELINES

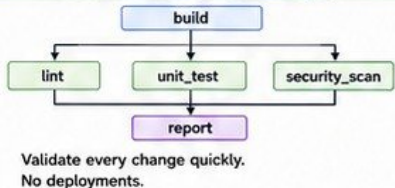
- ✓ Run independent jobs in parallel (test, lint, scan).
- ✓ Use `requires` to enforce only true dependencies.
- ✓ Fail fast: run quick checks first.
- ✓ Cache dependencies to reduce build time.
- ✓ Split long tests and run in parallel.
- ✓ Use workspaces to share data between jobs.
- ✓ Deploy only after tests and approvals succeed.
- ✓ Keep jobs small, focused and idempotent.
- ✓ Use filters (branches, tags) to avoid unnecessary runs.

💡 GOAL

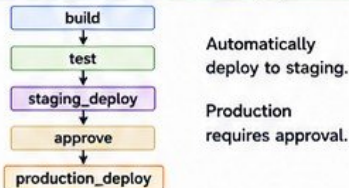
Fast feedback,
high reliability,
low cost,
and safe
deployments.

5.6 WORKFLOW PATTERNS

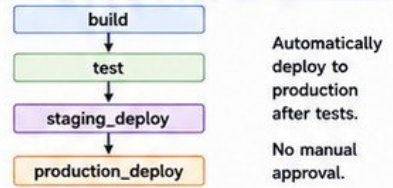
CONTINUOUS INTEGRATION (CI)



CONTINUOUS DELIVERY (CD)



CONTINUOUS DEPLOYMENT



COMMON MISTAKES

- Using `requires` for unrelated jobs (adds unnecessary wait).
- Running everything sequentially (slows down pipeline).
- Large monolithic jobs (hard to debug and slow to retry).



SENIOR ENGINEER TIP

- Design for speed without sacrificing reliability.
- Think in terms of dependency graph, not a straight line.
- Optimize the slowest 20% of your pipeline — it gives 80% of the gain.



PIPELINE PARAMETERS AND DYNAMIC CONFIGURATION

FLEXIBLE. DYNAMIC. AUTOMATED.

1. PIPELINE PARAMETERS

Parameters allow you to pass values into your pipeline at run time to customize behavior.



EXAMPLES

- Environment (dev, stage, prod)
- Region (us-east-1, eu-west-1)
- Image tag or version
- Feature flags
- Deploy components

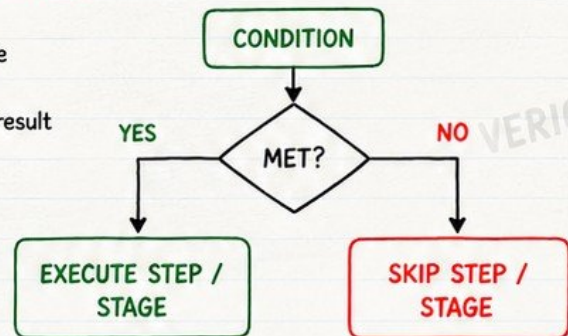
```
parameters:
  environment:
    type: string
    default: dev
  region:
    type: string
    default: us-east-1
  image_tag:
    type: string
    default: latest
```

2. CONDITIONAL EXECUTION

Run stages, jobs or steps only when conditions are met.

COMMON CONDITIONS

- Branch name
- Parameter value
- File changes
- Previous stage result

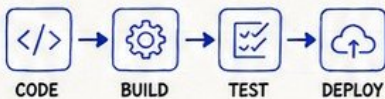


3. SETUP WORKFLOWS

Use parameters and dynamic configuration to set up workflows that adapt to different scenarios.

BEST PRACTICES

- Define clear parameters
- Provide sensible defaults
- Validate inputs
- Keep logic simple and readable
- Reuse with templates



4. DYNAMIC CONFIGURATION

Generate parts of your pipeline at runtime using parameters, templates or external data.

TECHNIQUES

- Parameterized templates
- Include/extend templates
- Matrix strategies
- Dynamic job generation
- External configuration files



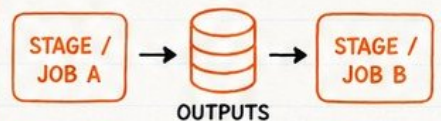
```
strategy:
  matrix:
    service: [api, web, worker]
    region: [us-east-1, eu-west-1]
  max-parallel: 4
```

5. CONTINUATION

Pass data between stages and workflows to drive decisions and maintain context.

COMMON METHODS

- Output variables
- Artifacts
- Runtime parameters
- State files

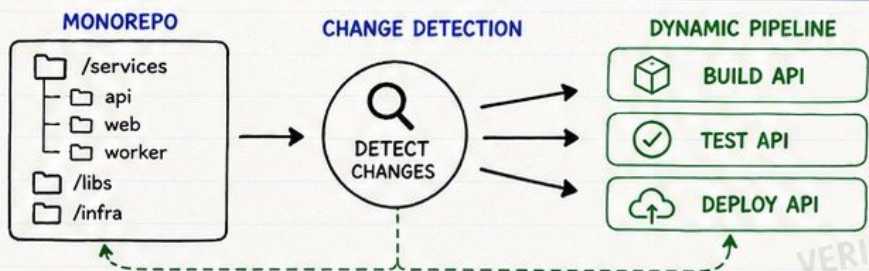


6. MONOREPO USE CASES

Dynamic pipelines are essential for monorepos to build and deploy only what changed.

COMMON PATTERNS

- Detect changed directories
- Map changes to services
- Run only affected tests and builds
- Deploy only changed services
- Use matrix for multiple services



KEY TAKEAWAY

Parameters and dynamic configuration make pipelines flexible, reusable and intelligent. They enable automation that adapts to your code, your teams and your business.



PIPELINE PARAMETERS AND DYNAMIC CONFIGURATION

FLEXIBLE. DYNAMIC. AUTOMATED.

1. PIPELINE PARAMETERS



Parameters allow you to pass values into your pipeline at run time to customize behavior.

EXAMPLES

- Environment (dev, stage, prod)
- Region (us-east-1, eu-west-1)
- Image tag or version
- Feature flags
- Deploy components

```
parameters:
  environment:
    type: string
    default: dev
  region:
    type: string
    default: us-east-1
  image_tag:
    type: string
    default: latest
```

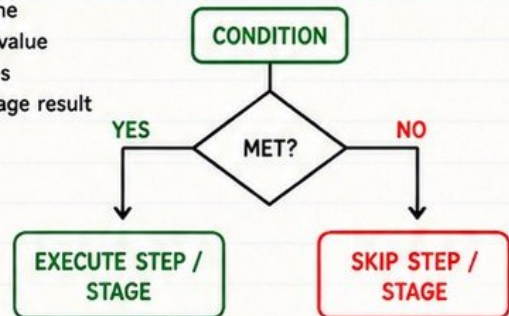
2. CONDITIONAL EXECUTION



Run stages, jobs or steps only when conditions are met.

COMMON CONDITIONS

- Branch name
- Parameter value
- File changes
- Previous stage result



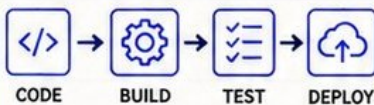
3. SETUP WORKFLOWS



Use parameters and dynamic configuration to set up workflows that adapt to different scenarios.

BEST PRACTICES

- Define clear parameters
- Provide sensible defaults
- Validate inputs
- Keep logic simple and readable
- Reuse with templates



4. DYNAMIC CONFIGURATION

Generate parts of your pipeline at runtime using parameters, templates or external data.

TECHNIQUES

- Parameterized templates
- Include/extend templates
- Matrix strategies
- Dynamic job generation
- External configuration files

```
strategy:
  matrix:
    service: [api, web, worker]
    region: [us-east-1, eu-west-1]
    max-parallel: 4
```

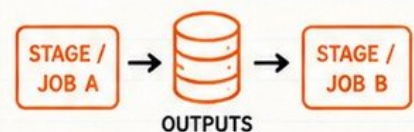
5. CONTINUATION



Pass data between stages and workflows to drive decisions and maintain context.

COMMON METHODS

- Output variables
- Artifacts
- Runtime parameters
- State files

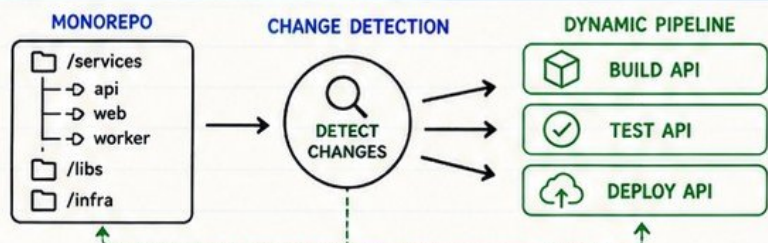


6. MONOREPO USE CASES

Dynamic pipelines are essential for monorepos to build and deploy only what changed.

COMMON PATTERNS

- Detect changed directories
- Map changes to services
- Run only affected tests and builds
- Deploy only changed services
- Use matrix for multiple services



KEY TAKEAWAY

Pipeline parameters and dynamic configuration give you the flexibility to build smarter, more adaptable CI/CD pipelines. Use them to control execution, tailor workflows, and scale across your organization.



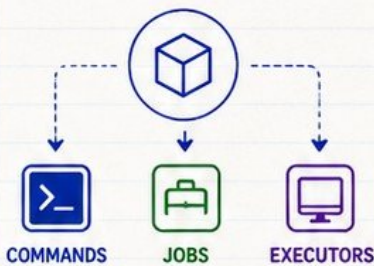
CIRCLECI ORBS

REUSABLE. SHAREABLE. POWERFUL.

1. WHAT ORBS ARE

Orbs are reusable packages of CircleCI configuration written in YAML.

They help you share and use: **commands**, **jobs**, and **executors**.



★ Orbs promote DRY, consistency and speed across your pipelines.

2. COMMANDS, JOBS & EXECUTORS

COMMANDS

- Reusable steps
- Run inside jobs
- Accept parameters

```
commands:
  say-hello:
    parameters:
      name:
        type: string
    steps:
      - run: echo "Hello << parameters.name >>"
```

JOBS

- Group commands
- Run as a unit
- Accept parameters

```
jobs:
  build-and-test:
    docker:
      - image: cimg/node:20.11
    steps:
      - checkout
      - say-hello:
          name: "CircleCI"
```

EXECUTORS

- Define the environment
- Reusable across jobs

```
executors:
  node-executor:
    docker:
      - image: cimg/node:20.11
```

3. USING PUBLIC ORBS

Public orbs are available on the CircleCI Orb Registry.

Add to your config:

```
orbs:
  node: circleci/node@5.1.0
```

Use in a job:

```
jobs:
  build:
    docker:
      - image: cimg/node:20.11
    steps:
      - checkout
      - node/install
      - node/test
```

★ Find orbs at:
<https://circleci.com/developer/orbs>

4. PRIVATE ORBS

Create orbs for your organization and keep them private.

Steps:

- 1 Create orb using `circleci orb init`
- 2 Develop and test locally
- 3 Publish to your namespace
- 4 Reference in your projects

Example:

```
orbs:
  my-orb: my-org/my-orb@1.0.0
```

★ Private orbs live in your CircleCI namespace.

5. ORB VERSIONING

Always pin to a specific version for reliability.

Version formats:

- Semantic versioning: 1.2.3
- Major: 1.x.x (breaking changes)
- Minor: 1.2.x (new features)
- Patch: 1.2.3 (bug fixes)

Best practice:

```
orbs:
  my-orb: my-org/my-orb@1.2.3
```

★ Avoid floating versions (e.g., @1.2 or @latest) in production.

6. SECURITY CONSIDERATIONS

- Use orbs from trusted sources only.
- Review orb source code if possible.
- Pin to specific versions.
- Limit orb access using context and least privilege.
- Scan for vulnerabilities in orb dependencies.
- Monitor orb updates and changelogs.

★ Good security + good versions = Safe and stable pipelines.

HOW ORBS WORK IN A PIPELINE

1. IMPORT ORB



Add orb to your config

2. USE COMPONENTS



Use commands, jobs and executors

3. RUN PIPELINE



Execute pipeline with reusable logic

4. SHARE & REUSE



Share across projects and teams

DEPENDENCY CACHING

SPEED UP BUILDS. REDUCE COSTS. IMPROVE RELIABILITY.

1. RESTORE_CACHE

Download and restore a cache before running steps.



PURPOSE

Reuse previously saved files to skip expensive downloads or builds.

EXAMPLE

```
- restore_cache:
  keys:
    - v1-deps-{{ .Branch }}-{{ checksum
      "package-lock.json" }}
    - v1-deps-{{ .Branch }}-
    - v1-deps-
```



Tries keys in order until a match is found.

2. SAVE_CACHE

Save files or directories to the cache for future use.



PURPOSE

Store dependencies after installation so future builds can restore them.

EXAMPLE

```
- save_cache:
  key: v1-deps-{{ .Branch }}-{{ checksum
    "package-lock.json" }}
  paths:
    - node_modules
    - ~/.npm
```



Run this after dependencies are installed successfully.

3. CACHE KEYS

Cache keys identify and version your cached data.



BEST PRACTICES

- Use specific keys first
- Add less specific fallbacks
- Include branch or OS when needed
- Use checksums for accuracy
- Version your cache (e.g., v1, v2)

EXAMPLE KEY STRATEGY

```
v1-deps-{{ checksum "package-lock.json" }}
v1-deps-{{ .Branch }}-{{ checksum
  "package-lock.json" }}
v1-deps-{{ .Branch }}-
v1-deps-
```



Specific → Less specific → Default key.

4. DEPENDENCY CHECKSUMS

Use file checksums to automatically invalidate caches when dependencies change.



WHY IT MATTERS

When dependency files change, the checksum changes → new cache key → fresh cache.

EXAMPLE

```
checksum "package-lock.json"
checksum "Pipfile.lock"
checksum "go.sum"
checksum "Gemfile.lock"
```



Checksums ensure you never use outdated dependencies.

5. CACHE INVALIDATION

Ensure old or incompatible caches are not used.



WAYS TO INVALIDATE

- Change the version in your key (v1 → v2)
- Update checksum when files change
- Clear caches manually in CircleCI
- Use branch or environment in key
- Set expiration policies (organization level)

EXAMPLE

```
# Bump version when you make changes
key: v2-deps-{{ checksum "package-lock.json" }}
```



New key = new cache. Old cache is ignored.

6. AVOIDING STALE CACHES

Stale caches can cause hard-to-debug failures and inconsistent builds.



BEST PRACTICES

- Always use checksums for dependency files
- Version your caches
- Scope keys by branch, OS, or image
- Keep cache contents small and focused
- Regularly clear old caches
- Test builds after dependency updates



Fresh cache = reliable builds. Clean cache = fewer surprises.

TYPICAL CACHING WORKFLOW

1. RESTORE CACHE



Try to restore cache using ordered keys

2. INSTALL DEPS



Install dependencies if cache miss

3. RUN BUILD & TESTS



Build, test, and verify your application

4. SAVE CACHE



Save cache for future builds



KEY TAKEAWAY

Caching reduces build time and costs, but only when done right. Use smart keys, checksums, and good practices to keep your pipelines fast and reliable.



WORKSPACES AND ARTIFACTS

— SHARE DATA. SHARE OUTPUTS. BUILD FASTER. —

1. PERSIST_TO_WORKSPACE

Save files or directories to a workspace so other jobs in the same workflow can use them.



PURPOSE

Share data between jobs.

EXAMPLE

```
- persist_to_workspace:  
  root: .  
  paths:  
    - node_modules  
    - dist  
    - build
```



Data is available to other jobs in the same workflow only.

2. ATTACH_WORKSPACE

Attach a workspace previously saved by another job.



PURPOSE

Retrieve shared data.

EXAMPLE

```
- attach_workspace:  
  at: .
```



Must be used in downstream jobs to access workspace data.

3. STORE_ARTIFACTS

Store files as artifacts to be downloaded from the job summary in CircleCI.



PURPOSE

Share build outputs with people or systems outside the workflow.

EXAMPLE

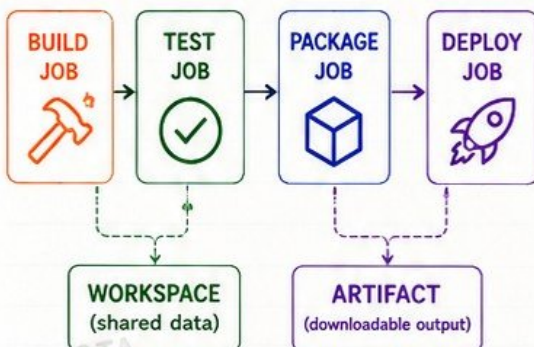
```
- store_artifacts:  
  path: dist/my-app.zip  
  destination: my-app.zip
```



Artifacts are retained even after the workflow completes.

4. SHARING BUILD OUTPUTS

Use workspaces to pass data between jobs.
Use artifacts to provide downloadable outputs.



5. WORKSPACE VERSUS CACHE VERSUS ARTIFACT

FEATURE	WORKSPACE	CACHE	ARTIFACT
Scope	Within the same workflow	Across workflows and branches	Outside the workflow (downloadable)
Purpose	Share files between jobs	Speed up dependency reinstallation	Share build outputs with users
Persistence	Temporary (workflow lifecycle)	Persistent until invalidated/evicted	Persistent (retention configurable)
Access	Only downstream jobs	Any future job/run	Anyone with access to the build
Size Recommendation	Small to medium (GBs)	Small to large (GBs)	Small to large (GBs)
Use When	Passing build outputs, dependencies between jobs	Dependencies rarely change and cost time to install	Binaries, reports, logs for distribution or audit

6. BEST PRACTICES

- Use workspaces for passing data between jobs in the same workflow.
- Use cache for dependencies that do not change often.
- Use artifacts for binaries, reports, and outputs shared outside the workflow.
- Keep workspace data small and only what is needed.
- Use specific keys and checksums for cache to avoid stale data.
- Clean up artifacts and set retention policies.
- Understand the difference to avoid unnecessary storage and slow builds.



WORKSPACE

Share within workflow



CACHE

Speed up future runs



ARTIFACT

Share outside workflow



KEY TAKEAWAY

Workspaces move data between jobs. Cache speeds up builds. Artifacts deliver outputs.
Use the right tool for the right job.



DOCKER IMAGE BUILD PIPELINES

BUILD ONCE. SHIP ANYWHERE.

1. BUILDING CONTAINER IMAGES

Build Docker images from your application source using a Dockerfile.



KEY POINTS

- Use a Dockerfile for repeatable builds
- Keep images small and secure
- Use multi-stage builds
- Only copy what you need

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine
WORKDIR /app
COPY --from=build /app/dist ./dist
CMD ["node", "dist/server.js"]
```

2. DOCKER LAYER CACHING

Leverage Docker layer caching to speed up subsequent builds.



BEST PRACTICES

- Order Dockerfile instructions from least to most frequently changing
- Copy dependency files first
- Use .dockerignore to exclude unnecessary files

Layer 4	Application Code (Changes often)
Layer 3	Build Dependencies (Changes sometimes)
Layer 2	Node Modules (Changes less often)
Layer 1	Base Image (Changes rarely)

Builds faster with caching

3. IMAGE TAGGING

Use consistent tagging to identify and version your images.



COMMON TAGS

- latest – not recommended for prod
- <version> – semantic version
- <git-sha> – commit hash
- branch name – feature or env
- date – build date

```
my-app:1.2.0
my-app:1.2.0-3f4a7b2
my-app:feature-login
my-app:2025-05-20
```

4. REGISTRY AUTHENTICATION

Authenticate securely with your container registry.



EXAMPLE (AWS ECR)

```
aws ecr get-login-password \
  --region us-east-1 | \
  docker login --username AWS \
  --password-stdin \
  123456789012.dkr.ecr.us-east-1.
  amazonaws.com
```

5. PUSHING IMAGES

Push your tagged images to the remote registry.



EXAMPLE COMMANDS

```
docker tag my-app:1.2.0 \
  123456789012.dkr.ecr.us-east-1.
  amazonaws.com/my-app:1.2.0

docker push \
  123456789012.dkr.ecr.us-east-1.
  amazonaws.com/my-app:1.2.0
```

6. IMMUTABLE IMAGE STRATEGY

Treat images as immutable to ensure consistency and security.



BEST PRACTICES

- Never change an existing tag
- Always build a new image for changes
- Deploy by immutable tags or digests
- Scan images before pushing
- Use read-only registry permissions in production

TYPICAL DOCKER IMAGE BUILD PIPELINE WORKFLOW

1. CHECKOUT CODE



Get source code from repository

2. BUILD IMAGE



Build Docker image using Dockerfile

3. TAG IMAGE



Tag image with version, branch, or commit

4. AUTHENTICATE



Login to container registry securely

5. PUSH IMAGE



Push tagged image to remote registry

6. USE IMAGE



Deploy or use image in next pipeline stages



KEY TAKEAWAY

A reliable Docker image pipeline ensures fast builds, secure images, and consistent deployments. Use caching, tagging, and immutability for efficient and predictable CI/CD.



TESTING AND QUALITY GATES

TEST EARLY. TEST OFTEN. SHIP CONFIDENTLY.

1. UNIT TESTS



Verify individual components in isolation.

BEST PRACTICES

- Fast and reliable
- No external dependencies
- Run on every commit
- High test coverage

EXAMPLE (JEST)

```
npm test -- --ci --reporters=default
--coverage
// package.json
{
  "scripts": {
    "test": "jest"
  }
}
```

2. INTEGRATION TESTS



Verify interactions between services, APIs, or modules.

BEST PRACTICES

- Test real integrations
- Use test databases
- Clean up data after tests
- Run in isolated environments

EXAMPLE (API TEST)

```
curl -X GET https://api.test.local/
health | grep "OK"
```

3. TEST SPLITTING & PARALLELISM



Split tests and run in parallel to save time.

BENEFITS

- Faster feedback
- Better resource utilization
- Scales with test suite size

EXAMPLE (CIRCLECI)

```
parallelism: 4
steps:
  - run:
      command: npm test
      -- --ci --parallel
```

4. CODE COVERAGE



Measure how much of your code is tested.

BEST PRACTICES

- Track coverage over time
- Focus on meaningful coverage
- Don't chase 100% blindly
- Exclude generated code

EXAMPLE (JEST)

```
npm test -- --coverage
# Output in coverage/lcov.info
```

5. QUALITY GATES



Define thresholds that must be met for code to pass.
Fail the pipeline when quality is below standards.

COMMON QUALITY GATES

- Code Coverage $\geq 80\%$
- No Critical or Major vulnerabilities
- No new code smells
- All tests must pass
- Maximum allowed test failures = 0

METRIC	THRESHOLD (EXAMPLE)
Coverage	$\geq 80\%$
Vulnerabilities (Critical)	0
Vulnerabilities (High)	0
Code Smells (New)	0
Duplications	$\leq 3\%$
Test Success Rate	100%

6. FAILING PIPELINES CORRECTLY



Fail fast when quality checks are not met.
Do not allow bad code to move forward.

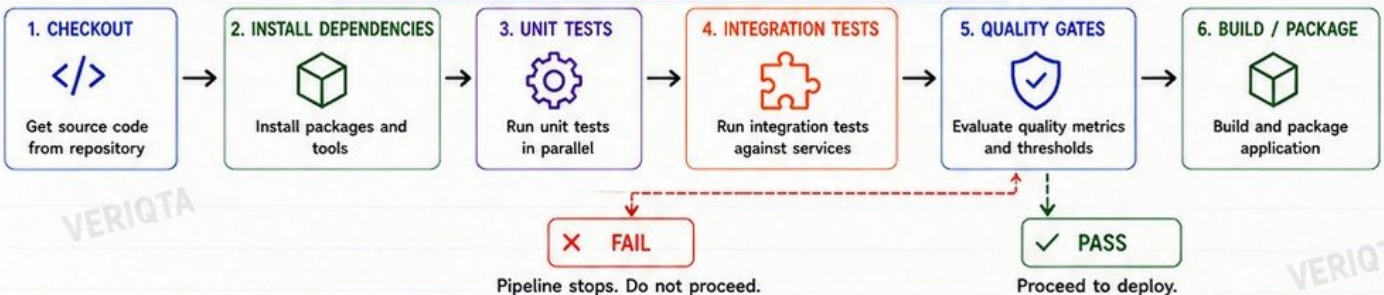
BEST PRACTICES

- Use exit codes to fail steps
- Fail on test failures or threshold violations
- Provide clear logs and error messages
- Block merge and deployment on failures

EXAMPLE (BASH)

```
if [ $COVERAGE -lt 80 ]; then
  echo "Coverage $COVERAGE% is below threshold"
  exit 1
fi
```

7. TYPICAL TESTING AND QUALITY GATE PIPELINE



KEY TAKEAWAY

Automated testing and quality gates ensure only reliable, secure, and maintainable code reaches production. Test early, enforce quality, and fail pipelines correctly to protect your users and your team.



SECRETS AND ENVIRONMENT VARIABLES

PROTECT SECRETS. SECURE PIPELINES. BUILD TRUST.

1. PROJECT VARIABLES

Store key-value variables at the project level for use in pipelines.

KEY POINTS

- Available to all pipelines
- Can be plaintext or masked
- Environment scoped
- Easy to manage

EXAMPLE

```
DB_HOST=localhost
API_URL=https://api.example.com
FEATURE_FLAG=true
```

2. CONTEXTS

Share environment variables and secrets across multiple projects securely.

KEY POINTS

- Centralized secret management
- Restrict access to specific users
- Apply to multiple projects
- Use for teams and environments

EXAMPLE

```
Context: production
- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- DOCKER_PASSWORD
```

3. SECRET MASKING

Mask sensitive values in logs to prevent accidental leaks.

BEST PRACTICES

- Enable masking for all secrets
- Avoid echoing secrets
- Use built-in masking patterns
- Regularly review variables

EXAMPLE

```
echo $DB_PASSWORD
Output: *****
(masked in logs)
```

4. ENVIRONMENT SEPARATION

Keep configurations and secrets separated by environment.

BEST PRACTICES

- Separate dev, staging, prod
- Use different contexts
- Avoid sharing secrets
- Test with realistic data

EXAMPLE

```
dev-context
staging-context
prod-context
```

5. LEAST PRIVILEGE

Grant only the minimum access required for jobs and users.

BEST PRACTICES

- Limit context access
- Use read-only credentials
- Avoid admin-level tokens
- Review permissions regularly

EXAMPLE

```
User A → dev-context (read-only)
User B → prod-context (deploy only)
Job → IAM role with least privilege
```

6. SECRET ROTATION

Rotate secrets regularly to reduce risk and limit exposure.

BEST PRACTICES

- Rotate on schedule
- Rotate after personnel changes
- Automate rotation when possible
- Revoke old secrets immediately

EXAMPLE

```
Old Key → Revoke
New Key → Update in Context
Validate → Deploy → Confirm
```

7. PREVENTING CREDENTIAL LEAKAGE

Prevent secrets from being exposed in code, logs, or artifacts.

BEST PRACTICES

- Never commit secrets to repository
- Use .gitignore for local secret files
- Avoid printing secrets in scripts
- Scan code for leaked secrets
- Use secure third-party actions

EXAMPLE (.gitignore)

```
.env
secrets.yml
*.pem
config/*.key
```

8. HOW SECRETS FLOW IN CIRCLECI

1. STORE



Store secrets in Project Variables or Contexts

2. ATTACH



Attach Context to pipeline or workflow

3. USE



Access secrets in jobs using environment vars

4. MASK



Secrets are masked in logs and outputs

5. SECURE



Pipelines run securely with least privilege access

6. ROTATE



Rotate secrets regularly and revoke old ones



KEY TAKEAWAY

Manage secrets and environment variables securely to protect your applications and infrastructure. Use contexts, follow least privilege, and rotate secrets regularly. Security in CI/CD is everyone's responsibility.



AWS DEPLOYMENT WITH CIRCLECI

BUILD. TEST. DEPLOY. SCALE ON AWS.

1. AWS AUTHENTICATION

Authenticate CircleCI securely to access AWS resources.



KEY METHODS

- AWS Access Keys
- Environment Variables
- AWS CLI Configuration
- OpenID Connect (OIDC) – Recommended

EXAMPLE (AWS CLI SETUP)

```
aws configure set aws_access_key_id $AWS_ACCESS_KEY_ID
aws configure set aws_secret_access_key $AWS_SECRET_ACCESS_KEY
aws configure set region $AWS_REGION
```

2. IAM ROLES AND PERMISSIONS

Use least privilege IAM roles for secure deployments.



BEST PRACTICES

- Create dedicated IAM role for CI/CD
- Attach managed or custom policies
- Limit permissions to required actions
- Use OIDC to assume role (no long-lived keys)
- Regularly audit permissions

EXAMPLE POLICY (S3 DEPLOY)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject", "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

3. DEPLOYING APPLICATIONS

Automate application deployment to AWS environments.



COMMON STRATEGIES

- Deploy static files to S3
- Deploy containers to ECS or EKS
- Use CodeDeploy for EC2 instances
- Blue/Green or Rolling deployments
- Environment promotion (dev → prod)

EXAMPLE (DEPLOY STEP)

```
- run:
  name: Deploy to S3
  command: |
    aws s3 sync ./build s3://my-bucket \
    --delete --cache-control "max-age=31536000"
```

4. S3 DEPLOYMENT

Deploy static websites and assets to Amazon S3.



BEST PRACTICES

- Enable versioning
- Set proper bucket policies
- Use CloudFront for CDN
- Enable encryption (SSE-S3/SSE-KMS)
- Invalidate cache after deployment

EXAMPLE COMMAND

```
aws s3 sync ./dist s3://my-website-bucket \
--delete --acl public-read \
--cache-control "max-age=31536000"
```

5. ECS / EKS INTEGRATION CONCEPTS

Deploy containerized applications to ECS or EKS clusters.



KEY CONCEPTS

- Push image to ECR
- Update ECS service or EKS deployment
- Use task definitions / k8s manifests
- Rolling updates for zero downtime
- Monitor deployment status

EXAMPLE (ECS DEPLOY)

```
aws ecs update-service \
--cluster my-cluster \
--service my-service \
--force-new-deployment
```

6. SECURE CLOUD AUTHENTICATION

Secure authentication prevents credential leakage and protects your cloud.



BEST PRACTICES

- Use OIDC + IAM roles (no long-term keys)
- Store secrets in CircleCI contexts
- Mask sensitive values in logs
- Rotate credentials regularly
- Restrict IP ranges and permissions

OIDC BENEFITS

- No static AWS keys
- Short-lived tokens
- Stronger security posture

7. TYPICAL AWS DEPLOYMENT WORKFLOW IN CIRCLECI



KEY TAKEAWAY

CircleCI + AWS enables secure, automated, and scalable deployments. Use IAM roles, least privilege, and OIDC for secure authentication. Automate everything, ship confidently, and monitor continuously.



KUBERNETES DEPLOYMENT PIPELINES

AUTOMATE. DEPLOY. VERIFY. SCALE.

1. KUBECTL DEPLOYMENT

Deploy applications using kubectl commands in your pipeline.



KEY COMMANDS

- Apply manifests
- Set images
- Rolling update

EXAMPLE

```
kubectl apply -f k8s/  
kubectl set image deployment/  
my-app my-app=$IMAGE_TAG  
--namespace production  
kubectl rollout status  
deployment/my-app  
--namespace production
```

2. HELM DEPLOYMENT

Use Helm charts for repeatable and manageable deployments.



KEY COMMANDS

- Install / upgrade release
- Set values
- Manage versions

EXAMPLE

```
helm upgrade --install my-app \  
./chart/ -n production \  
--set image.tag=$IMAGE_TAG \  
--atomic --timeout 5m  
helm list -n production
```

3. KUBERNETES AUTHENTICATION

Authenticate securely to your Kubernetes cluster.



BEST PRACTICES

- Use service accounts
- Prefer short-lived tokens
- Use OIDC with cloud provider
- Store credentials in contexts

EXAMPLE (KUBECONFIG)

```
kubectl config use-context  
prod-cluster  
kubectl config set-context  
prod-cluster --namespace  
production
```

4. NAMESPACE SEPARATION



Isolate environments using Kubernetes namespaces.

BEST PRACTICES

- Separate dev, staging, prod
- Apply ResourceQuotas
- Use NetworkPolicies
- Limit cross-namespace access

EXAMPLE

```
apiVersion: v1  
kind: Namespace  
metadata:  
  name: production  
  labels:  
    environment: production
```

5. DEPLOYMENT VERIFICATION

Verify that your application is running as expected.



VERIFICATION CHECKS

- Pods are running
- Services are available
- Endpoints are healthy
- Application health checks

EXAMPLE COMMANDS

```
kubectl get pods -n production  
kubectl get svc -n production  
kubectl get endpoints -n production  
kubectl describe deployment my-app  
-n production
```

6. ROLLOUT STATUS

Monitor rollout progress to ensure a successful deployment.



KEY COMMANDS

- Check rollout status
- View rollout history
- Monitor updates

EXAMPLE COMMANDS

```
kubectl rollout status deployment/  
my-app -n production  
kubectl rollout history deployment/  
my-app -n production  
kubectl get rollout deployment/my-app  
-n production
```

7. ROLLBACK STRATEGY

Rollback to a previous stable version when needed.



BEST PRACTICES

- Keep a deployment history
- Use health checks and readiness probes
- Automate rollback on failure
- Test rollback process regularly

EXAMPLE COMMANDS

```
kubectl rollout undo deployment/my-app  
-n production  
kubectl rollout undo deployment/my-app  
--to-revision=2 -n production  
kubectl rollout status deployment/my-app  
-n production
```

8. TYPICAL KUBERNETES DEPLOYMENT PIPELINE WORKFLOW



KEY TAKEAWAY

Kubernetes deployment pipelines ensure reliable, repeatable, and secure application delivery. Use proper authentication, namespace isolation, and rollback strategies to deploy with confidence and minimize downtime.



APPROVAL GATES AND PRODUCTION RELEASES

CONTROL CHANGES. PROTECT PRODUCTION. DELIVER VALUE.

1. MANUAL APPROVAL JOBS

Pause the pipeline and wait for human approval before continuing.



KEY POINTS

- Add approval steps before sensitive stages
- Require explicit human action
- Assign approvers or approval groups
- Set timeouts to avoid indefinite waits

EXAMPLE (CIRCLECI)

```
release-prod:
  type: approval
  requires:
    - deploy-staging
  filters:
    branches:
      only: main
  approval_timeout: 1h
```

2. ENVIRONMENTS FLOW

Promote changes through environments in a controlled way.



BEST PRACTICES

- Develop in isolation
- Test thoroughly in staging
- Approve and release to production
- Promote the same artifact
- Maintain environment parity

EXAMPLE FLOW

```
Development (dev)
→ Build & Test
→ Deploy to Staging
→ Approval Gate
→ Deploy to Production
→ Monitor & Verify
```

3. RELEASE CONTROLS

Control how and when changes are released to production.



BEST PRACTICES

- Restrict who can approve releases
- Use branch and tag protections
- Limit deployment windows
- Use feature flags for gradual release
- Keep rollback plan ready

EXAMPLE (DEPLOY WINDOW)

```
filters:
  branches:
    only: main
triggers:
  - schedule:
      cron: "0 2-4 * * 1-5"
    filters:
      branches:
        only: main
```

4. CHANGE MANAGEMENT

Track, document, and communicate changes properly.



KEY ACTIVITIES

- Document change purpose and impact
- Link work to tickets or change records
- Communicate before and after release
- Maintain audit trail of deployments
- Review and learn from each change

EXAMPLE (CHANGE LOG ENTRY)

```
Change ID: CHG-2025-05-20-001
Summary: Add payment retry logic
Type: Feature
Risk: Medium
Approver: Release Manager
Status: Approved
```

5. PRODUCTION PROTECTION

Add layers of protection to reduce risk in production.



BEST PRACTICES

- Require approval for production deploys
- Use protected branches and tags
- Limit direct access to production
- Use read-only by default
- Monitor and alert on changes

EXAMPLE (PROTECTED BRANCH)

```
Branch: main
Protection Rules:
- Require pull request
- Require status checks
- Require approvals (min 1)
- Dismiss stale reviews
- Restrict pushes to admins
```

6. DEPLOYMENT AUTHORIZATION

Ensure only authorized people or teams can deploy to production.



BEST PRACTICES

- Use approval groups or roles
- Apply least privilege
- Enable 2-person approval if needed
- Log who approved and when
- Revoke access when not needed

EXAMPLE (APPROVAL GROUP)

```
contexts:
  - name: prod-approval-group
    required: true
    users:
      - alice
      - bob
      - release-manager
```

7. TYPICAL APPROVAL AND PRODUCTION RELEASE WORKFLOW



KEY TAKEAWAY

Approval gates and release controls reduce risk and ensure only high-quality changes reach production. Use manual approvals, environment promotion, and least privilege to protect your production systems and maintain customer trust.



PERFORMANCE AND PIPELINE OPTIMIZATION

FASTER PIPELINES. LOWER COSTS. HIGHER IMPACT.

1. PARALLELISM

Run independent jobs at the same time to reduce total pipeline time.



BEST PRACTICES

- Identify independent tasks
- Split long workflows
- Use parallel steps in jobs
- Avoid unnecessary dependencies

EXAMPLE (CIRCLECI)

```
version: 2.1
jobs:
  build:
    parallelism: 4
    steps:
      - checkout
      - run: make build
```

2. TEST SPLITTING

Split tests across multiple machines to run faster.



BEST PRACTICES

- Split by timing or historical data
- Balance test duration
- Use dynamic test splitting
- Re-run failed tests only

EXAMPLE (CIRCLECI)

```
version: 2.1
jobs:
  test:
    parallelism: 4
    steps:
      - checkout
      - run: circleci tests split
      - run: circleci tests run
```

3. CACHING STRATEGY

Cache dependencies and build outputs to avoid re-downloading or re-building.



BEST PRACTICES

- Cache dependencies
- Use checksum-based keys
- Restore before install
- Save cache only when needed

EXAMPLE (CACHE YAML)

```
- restore_cache:
  keys:
    - v1-deps-{{ checksum "pom.xml" }}
    - v1-deps-
- save_cache:
  key: v1-deps-{{ checksum "pom.xml" }}
  paths:
    - ~/.m2
```

4. RESOURCE CLASSES

Use the right machine size for the job. More power when needed, less when not.



BEST PRACTICES

- Use small for simple tasks
- Use medium/large for builds
- Use GPU only when required
- Match resources to workload

EXAMPLE

```
jobs:
  build:
    docker:
      - image: cimg/base:stable
        resource_class: medium
  heavy-build:
    docker:
      - image: cimg/base:stable
        resource_class: large
```

5. WORKFLOW CONCURRENCY

Limit how many workflows run at the same time to avoid resource contention and reduce wait time.



BEST PRACTICES

- Set concurrency limits
- Prioritize critical workflows
- Use contexts and filters
- Avoid queue build up

EXAMPLE (CONFIG.YML)

```
workflows:
  version: 2
  build_and_deploy:
    jobs:
      - build
      - deploy:
          requires:
            - build
```

6. REDUCING PIPELINE DURATION

Optimize every step to remove delays and speed up feedback.



BEST PRACTICES

- Fail fast on errors
- Combine steps where possible
- Use lightweight images
- Avoid redundant work
- Optimize dependency install

EXAMPLE (FAIL FAST)

```
- run: npm test -- --bail --ci
- run: make lint || exit 1
- run: make build
```

7. CONTROLLING CI COSTS

Optimize resource usage and reduce unnecessary consumption.



BEST PRACTICES

- Right-size resource classes
- Use caching to reduce minutes
- Limit concurrency
- Cancel redundant builds
- Use self-hosted runners when viable

COST SAVING TIPS

- Avoid idle time
- Avoid over-provisioning
- Monitor usage regularly
- Set workflow triggers carefully

8. OPTIMIZATION CHECKLIST

- ☒ Maximize Parallelism
- ☒ Split Tests Intelligently
- ☒ Use Effective Caching
- ☒ Right-size Resources
- ☒ Limit Concurrency
- ☒ Reduce Pipeline Duration
- ☒ Monitor and Control Costs
- ☒ Continuously Improve

9. PIPELINE OPTIMIZATION FLOW

1. ANALYZE



Identify bottlenecks and slow steps

2. PARALLELIZE



Run independent tasks in parallel

3. SPLIT TESTS



Distribute tests across multiple machines

4. CACHE SMARTLY



Cache dependencies and build outputs

5. OPTIMIZE RESOURCES



Use the right size for every job

6. MONITOR & IMPROVE



Measure, learn, and continuously optimize



KEY TAKEAWAY

A fast and efficient pipeline delivers rapid feedback, reduces costs, and boosts developer productivity. Optimize continuously to get the most out of your CI/CD investment.



SECURITY AND SUPPLY CHAIN PROTECTION

SECURE TODAY. PROTECT TOMORROW.

1. LEAST-PRIVILEGE CREDENTIALS

Grant only the minimum access needed for each job and tool.



BEST PRACTICES

- Use project-level environment variables
- Create dedicated service accounts
- Avoid using personal tokens
- Rotate credentials regularly
- Review and revoke unused access

```
# Example: Limit AWS permissions
policies:
  - Statement:
    - Effect: Allow
      Action:
        - s3:PutObject
        - s3:PutObjectAcl
      Resource: arn:aws:s3:::my-bucket/*
```

2. DEPENDENCY SECURITY

Scan and manage dependencies to prevent vulnerable libraries from reaching production.



BEST PRACTICES

- Use lock files and pin versions
- Scan dependencies in every build
- Monitor for newly disclosed CVEs
- Update dependencies proactively
- Fail builds on critical vulnerabilities

```
# Example: npm audit
npm ci
npm audit --audit-level=high
# or
 snyk test --severity-threshold=high
```

3. CONTAINER SCANNING

Scan container images for OS packages and known vulnerabilities before pushing.



BEST PRACTICES

- Scan base images and final images
- Fail builds on high/critical issues
- Use trusted scanners
- Keep base images minimal
- Rescan when base images update

```
# Example: Trivy scan
trivy image --exit-code 1 \
--severity HIGH,CRITICAL my-app:1.2.0
```

4. SECRET MANAGEMENT

Store and access secrets securely. Never hardcode secrets in code or configuration.



BEST PRACTICES

- Use CircleCI Contexts or external vaults
- Mask secrets in logs
- Limit secret access to specific jobs
- Rotate secrets regularly
- Avoid echoing secrets in scripts

```
# Example: Use context environment
contexts:
  - org-global-secrets
# Access secret as environment variable
$ DATABASE_PASSWORD
```

5. THIRD-PARTY ORB RISKS

Orbs run code with your permissions. Treat third-party orbs with caution.



BEST PRACTICES

- Use orbs from trusted, verified sources
- Review orb source and permissions
- Pin orb versions
- Monitor orb updates and changelogs
- Prefer maintained, active orbs

```
# Pin orb version (example)
version: 2.1
orbs:
  aws-cli: circleci/aws-cli@2.1.0
```

6. ARTIFACT INTEGRITY

Ensure build artifacts and images are authentic, untampered, and traceable.



BEST PRACTICES

- Sign container images
- Verify checksums and signatures
- Use immutable tags or digests
- Store SBOM (Software Bill of Materials)
- Enforce provenance and attestations

```
# Example: Verify image by digest
docker pull my-app@sha256:abc123...
docker inspect my-app@sha256:abc123...
```

7. CI/CD SUPPLY CHAIN THREATS

Protect your pipeline from attacks that target your code, dependencies, tools, and infrastructure.

COMMON THREATS

- Malicious dependencies
- Compromised build tools or orbs
- Secret leakage
- Dependency confusion
- Build environment tampering

BEST PRACTICES

- Use trusted sources for code and dependencies
- Verify integrity of dependencies and tools
- Enable 2FA on all accounts
- Monitor and audit pipeline access
- Keep CI/CD tools and runners updated
- Implement branch protection and code reviews
- Detect and alert on suspicious pipeline activity



KEY TAKEAWAY

Security is everyone's responsibility. Build secure pipelines, protect your supply chain, and deliver trusted software with confidence.



CIRCLECI PRODUCTION TROUBLESHOOTING

FIND. FIX. SHIP.

1. FAILED BUILDS

Builds fail for many reasons. Identify the real cause quickly.



COMMON CAUSES

- Code errors
- Test failures
- Missing dependencies
- Misconfigurations
- Environment issues

TROUBLESHOOTING

- Read logs from top to bottom
- Check the first error
- Reproduce locally
- Validate environment parity

```
circleci local execute
--job build
circleci config validate
```

2. CACHE PROBLEMS

Caches can speed builds but also cause issues.



COMMON ISSUES

- Cache not restoring
- Stale or corrupted cache
- Wrong cache paths
- Cache key collisions

TROUBLESHOOTING

- Verify cache keys
- Check restore/save paths
- Add checksum to keys
- Clear cache when needed

```
- restore_cache:
  keys:
    - v1-deps-{{ checksum
      "package-lock.json" }}
    - v1-deps-
```

3. DOCKER FAILURES

Docker issues are a top cause of failed pipelines.



COMMON CAUSES

- Docker build errors
- Image pull failures
- Disk space issues
- Dockerfile mistakes
- Registry rate limits

TROUBLESHOOTING

- Check Dockerfile syntax
- Inspect build logs
- Verify base image access
- Prune unused images

```
docker system df -h
docker image prune -af
docker build --no-cache .
```

4. AUTHENTICATION ERRORS

Auth issues block access to registries, APIs, or resources.



COMMON CAUSES

- Expired tokens
- Wrong credentials
- Missing permissions
- Incorrect contexts

TROUBLESHOOTING

- Validate environment variables
- Re-authenticate to services
- Check permission scopes
- Rotate credentials securely

```
echo $CIRCLE_OIDC_TOKEN
aws sts get-caller-identity
gcloud auth list
```

5. RESOURCE EXHAUSTION

Jobs fail when resources are insufficient.



COMMON CAUSES

- High CPU usage
- Out of memory
- Disk space full
- Too many parallel jobs

TROUBLESHOOTING

- Monitor resource usage
- Increase resource_class
- Reduce parallelism
- Clean up workspace/cache

```
resource_class: large
docker system prune -af
df -h
```

6. WORKFLOW DEPENDENCY PROBLEMS

Jobs fail due to dependency or configuration issues.



COMMON CAUSES

- Missing requires
- Circular dependencies
- Wrong job names
- Conditional logic errors

TROUBLESHOOTING

- Verify requires relationships
- Check workflow syntax
- Validate job names
- Test conditional logic

```
workflows:
  build_and_test:
    jobs:
      - build
      - test:
          requires: [build]
```

7. KUBERNETES DEPLOYMENT FAILURES

Deployments fail due to cluster or manifest issues.



COMMON CAUSES

- Invalid manifests
- Image pull errors
- Readiness/Liveness failures
- Insufficient permissions

TROUBLESHOOTING

- Check kubectl logs
- Describe pods and events
- Verify image and secrets
- Rollback if necessary

```
kubectl get pods -n prod
kubectl describe pod <pod> -n prod
kubectl logs <pod> -n prod
```

8. DEBUGGING WITH SSH

Use SSH to debug complex pipeline issues.



HOW TO USE

- Enable SSH in job
- SSH into running container
- Inspect files and environment
- Reproduce and fix

```
# Enable SSH in config
- add_ssh_keys:
  fingerprints:
    - "SHA256:..."
```

```
ssh -o StrictHostKeyChecking=no \
circleci@<ip-address>
```

9. READING PIPELINE LOGS SYSTEMATICALLY

- 1 Start from the top and read everything.
- 2 Look for the first error or failure message.
- 3 Understand the context before the error.
- 4 Check environment variables and versions.
- 5 Expand collapsed sections for full details.
- 6 Use search (Ctrl + F) to find keywords.
- 7 Identify patterns in repeated failures.
- 8 Save important log sections for reference.



EXAMPLE LOG ANALYSIS

```
12:34:01 Starting: Checkout code
12:34:03 ✓ Completed: Checkout code
12:34:05 Starting: Build
12:34:20 ERROR: npm install failed
12:34:20 npm ERR! code ERESOLVE
12:34:20 npm ERR! ERESOLVE unable to resolve dependency tree
12:34:20 npm ERR! Fix the upstream dependency conflict
12:34:20 Failed: Build (exit code 1)
12:34:20 Job failed
```

KEY TAKEAWAY

Most CircleCI issues come down to configuration, credentials, environment, or resources. Stay systematic, read logs carefully, and isolate variables to fix problems faster.



REMEMBER

Observe → Identify → Understand → Fix → Verify → Improve
Consistency, monitoring, and security make pipelines reliable and production-ready.



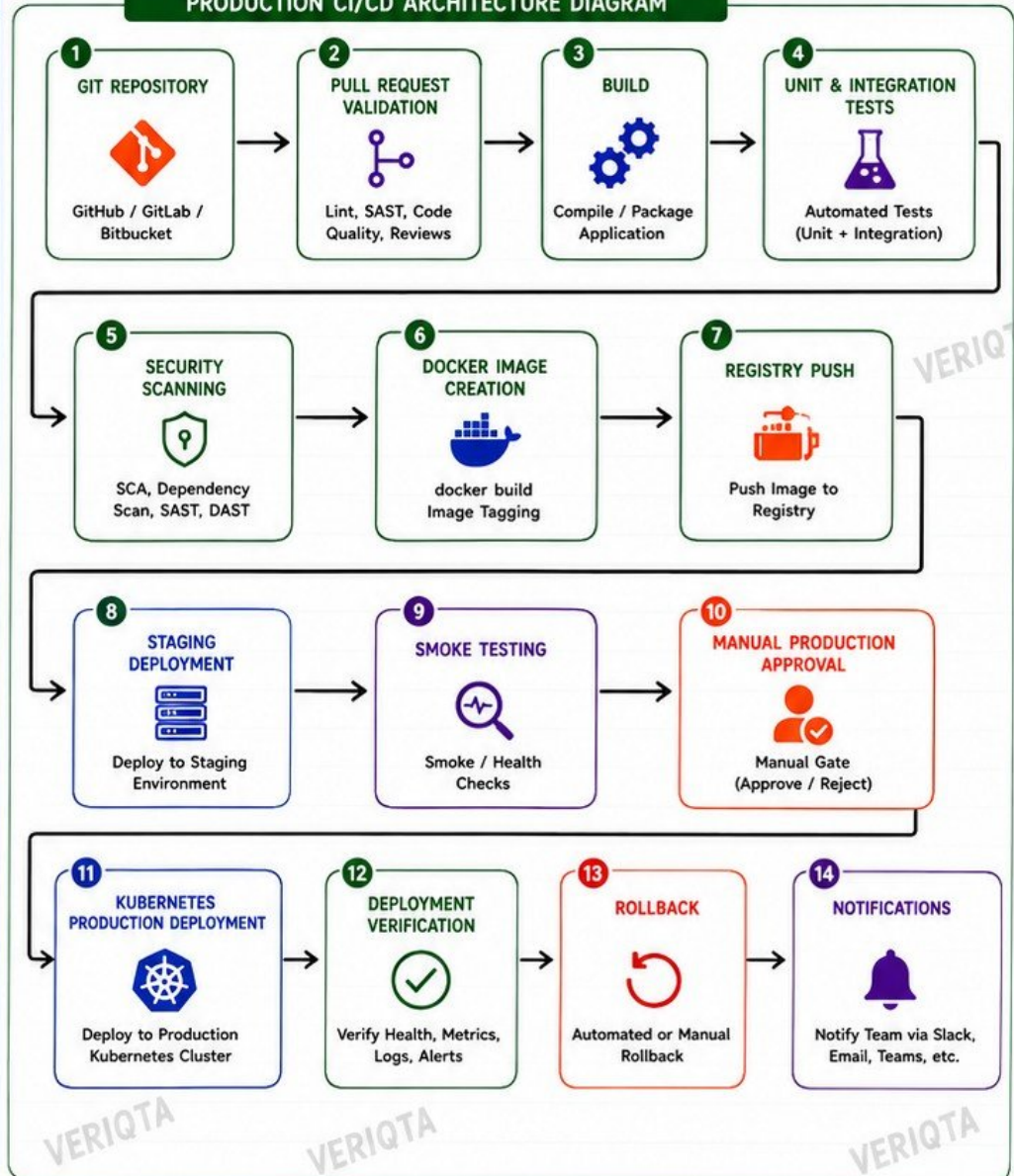
PRODUCTION CI/CD ARCHITECTURE PROJECT

BUILD RELIABLE PIPELINES. DELIVER VALUE. RUN IN PRODUCTION.

CI/CD PIPELINE STEPS

- Git Repository**
Source code is stored in Git.
- Pull Request Validation**
Code review and static checks.
- Build**
Compile and build the application.
- Unit & Integration Tests**
Run automated tests to validate functionality.
- Security Scanning**
Scan code and dependencies for vulnerabilities.
- Docker Image Creation**
Build Docker image from application.
- Registry Push**
Push image to container registry.
- Staging Deployment**
Deploy to staging environment.
- Smoke Testing**
Run smoke tests to verify staging deployment.
- Manual Production Approval**
Human approval required to deploy to production.
- Kubernetes Production Deployment**
Deploy to production Kubernetes cluster.
- Deployment Verification**
Verify application health in production.
- Rollback**
Rollback to previous version if issues detected.
- Notifications**
Notify team on success or failure.

PRODUCTION CI/CD ARCHITECTURE DIAGRAM



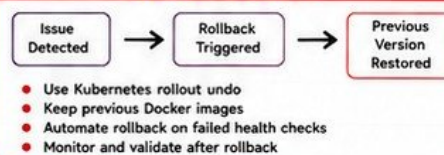
PIPELINE CHARACTERISTICS

- ✓ Automated, repeatable and auditable
- ✓ Secure by design with scanning at multiple stages
- ✓ Manual approval gate before production deployment
- ✓ Fast feedback and early failure detection
- ✓ Rollback ready for safe recovery
- ✓ Visibility and notifications for the entire team

TECHNOLOGIES (EXAMPLE)

- CI/CD: CircleCI
- Source Control: GitHub / GitLab
- Registry: Docker Hub / ECR / GCR
- Container: Docker
- Orchestration: Kubernetes
- Notifications: Slack / Email / Teams

ROLLBACK STRATEGY



KEY TAKEAWAY

A strong CI/CD architecture ensures secure, reliable and automated delivery. Every step adds quality, security and confidence to your production deployments.

